ФГАОУ ВО «НИУ ИТМО»
Факультет программной инженерии
и компьютерной техники

# Лабораторная работа №3

## Задача классификации

(по дисциплине «Системы искусственного интеллекта»)

**Выполнил:**

Лабин Макар Андреевич,
СИИ 2.1, P3331, 466449.

**Проверил:**

Запорожцев Иван Федорович,
кандидат технических наук

г. Санкт-Петербург, Россия
2026

# Содержание

**1 Задание** **1**

**2 Выполнение работы** **3**
2.1 Обучение моделей . . . . . 3
2.1.1 Выбор датасета . . . . . 3
2.1.2 Формулировка задачи . . . . . 3
2.1.3 Обучение LogisticRegression . . . . . 4
2.1.4 Обучение SVM . . . . . 5
2.2 Метрики многоклассовой классификации . . . . . 6
2.2.1 AUC-ROC . . . . . 6
2.2.1.1 Определения . . . . . 6
2.2.1.2 Ручной расчёт характеристик . . . . . 8
2.2.1.3 Визуализация ROC . . . . . 9
2.2.2 PR-кривая . . . . . 11
2.2.2.1 Определения . . . . . 11
2.2.2.2 Ручной расчёт характеристик . . . . . 11
2.2.3 Выводы . . . . . 11
2.3 Применение OneClassSVM . . . . . 11

**3 Источники литературы** **13**

# 1 Задание

**Task 1:** *[2 балла]*

Выберите датасет и задачу для него (классификация) — с Kaggle или UCI Machine Learning Repository.

Рекомендации:

- количество признаков — не менее 10;
- количество объектов — не менее 100;
- возраст датасета — не старее 2010 года;
- пользуйтесь датасетом из предыдущих лабораторных работ, если он подходит под условия выше.

Затем выполните следующие действия:

- Выберите (или сгенерируйте сами) признак, который будет целевым для бинарной классификации.
- Используйте LogisticRegression из sklearn. Задайте несколько параметров метода самостоятельно, поясните их назначение и причину выбора конкретных значений.
- Используйте несколько метрик, чтоб оценить результаты бинарной классификации (метрики, доступные в sklearn).

**Task 2:** *[2 балла]*

Выполните бинарную классификацию аналогично предыдущему пункту с использованием SVM из sklearn. Используйте `kernel='linear'`, сами подберите параметр регуляризации C и ответьте на вопрос:

- Может ли SVM учесть несбалансированность классов? Если да, то как? Если нет, то почему?

**Task 3:** *[6 баллов]*

(a) Многоклассовая классификация.[2 балла]

- Дайте определения для подходов: микро, макро, взвешенный, пообъектный.
- Выполните расчёты вручную для слайда 43 (на примере любых 3 промежуточных AUC ROC, поясните, как из промежуточных характеристик получаются 4 финальных).
- В рамках детализации **оцифруйте** ROC: в Excel, отрисуйте линиями в Word или отрисуйте отдельными отрезками Python-кодом (не используйте готовые функции для ROC).

(b) Для одного из кейсов (одного класса):[2 балла]

- Введите три порога бинаризации так, чтоб получились различные Accuracy, Recall, Precision.

- Вычислите эти метрики вручную и сопроводите визуализацией расчёта для Recall и Precision (см. пример из лекции с окружностями разных цветов и радиусов).
- По трём парам (Precision, Recall) постройте эскиз PR-кривой.

(c) Сделайте выводы по результатам расчётом AUC ROC для многоклассовой классификации:[1 балл]

- какой вариант лучше и почему;
- объясниь падение AUC ROC при переходе к многоклассовым.

(d) В каких кейсах/доменах следует применять те или иные варианты? Запишите выводы. [1 балл]

**Task 4:** *[2 балла]*

Исследуйте вопрос выбросов/новизны для данных в вашем датасете с использованием OneClassSVM. Дайте краткую теоретическую характеристику, используйте готовые решения для получения результатов, сделайте выводы.

Материалы для изучения указаны в списке литературы.[1; 2]

# 2 Выполнение работы

## 2.1 Обучение моделей

### 2.1.1 Выбор датасета

Для рассмотрения задачи регрессии был выбран датасет Student Performance с репозитория UCI Machine Learning.[3] Он имеет следующие особенности:

- количество признаков — 30;
- количество объектов — 649;
- год публикации — 2014;
- использовался в предыдущей лабораторной работе.[4]

В датасете представлены результаты студентов среднего образования двух португальских школ. Данные включают оценки студентов, демографические, социальные и школьные характеристики, и были собраны с использованием школьных отчётов и анкет.

Предоставлены два набора данных, касающиеся успеваемости по двум различным предметам: Математика и Португальский язык.

### 2.1.2 Формулировка задачи

В качестве целевой переменной *(метки класса)* предлагается использовать желание получить высшее образование (`higher`).

В качестве фичей *(предикторов)* возьмём следующие признаки из датасета:

- `age` (возраст) — числовой признак, который может коррелировать с желанием получить высшее образование;
- `Medu` (образование матери) — числовой признак, который может коррелировать с желанием получить высшее образование, так как более высокий уровень образования родителей может способствовать академическим амбициям студентов;
- `Fedu` (образование отца) — числовой признак, аналогично `Medu`;
- `absences` (количество пропусков) — числовой признак, который может отражать уровень ответственности и дисциплины студентов;
- `failures` (количество академических неудач) — числовой признак, который может быть связан с поведением студентов и их отношением к учебе;
- `G3` (итоговая оценка) — числовой признак, который может быть индикатором академической успеваемости и мотивации студентов;
- `famsup` (поддержка семьи) — категориальный признак, который может влиять на желание студентов продолжать образование;

- `schoolsup` (поддержка школы) — категориальный признак, который может отражать доступность ресурсов и поддержки для студентов;
- `studytime` (время, затраченное на учебу) — числовой признак, который может быть связан с академической мотивацией.

### 2.1.3 Обучение LogisticRegression

Для обучения модели логистической регрессии воспользуемся следующим скриптом, который указан в листинге 1.

Листинг 1: Обучение модели логистической регрессии.

```
logistic_regression_metrics = dict()

reg_log = LogisticRegression()

preprocessor = ColumnTransformer(
    transformers=[
        (
            'cat',
            OneHotEncoder(sparse_output=False),
            categorical_columns,
        ),
        (
            'num',
            StandardScaler(),
            numeric_columns,
        ),
    ],
    remainder='drop',
)

pipe_log = Pipeline(steps=[
    ("preprocessor", clone(preprocessor)),
    ("logistic", reg_log)
])

pipe_log.fit(train[predictors], train[target_column])

y_pred_log = pipe_log.predict(test[predictors])

logistic_regression_metrics['accuracy'] =
↪  accuracy_score(test[target_column], y_pred_log)
logistic_regression_metrics['precision'] =
↪  precision_score(test[target_column], y_pred_log, pos_label='yes')
logistic_regression_metrics['recall'] = recall_score(test[target_column],
↪  y_pred_log, pos_label='yes')
logistic_regression_metrics['f1'] = f1_score(test[target_column],
↪  y_pred_log, pos_label='yes')
```

В результате обучения модели логистической регрессии были получены следующие метрики, которые представлены в таблице 1.

Таблица 1: Метрики модели логистической регрессии.

| Метрика | Значение |
|---|---|
| Accuracy | 0.92344 |
| Precision | 0.93564 |
| Recall | 0.98438 |
| F1-score | 0.95939 |

### 2.1.4 Обучение SVM

Для обучения модели SVM воспользуемся следующим скриптом, который указан в листинге 2.

Листинг 2: Обучение модели SVM.

```
support_vector_machine_metrics = dict()

reg_svc = SVC(
    kernel='linear',
    C=C,
)

pipe_svc = Pipeline(steps=[
    ("preprocessor", clone(preprocessor)),
    ("svc", reg_svc)
])

pipe_svc.fit(train[predictors], train[target_column])

y_pred_svc = pipe_svc.predict(test[predictors])

support_vector_machine_metrics['accuracy'] =
↪  accuracy_score(test[target_column], y_pred_svc)
support_vector_machine_metrics['precision'] =
↪  precision_score(test[target_column], y_pred_svc, pos_label='yes')
support_vector_machine_metrics['recall'] =
↪  recall_score(test[target_column], y_pred_svc, pos_label='yes')
support_vector_machine_metrics['f1'] = f1_score(test[target_column],
↪  y_pred_svc, pos_label='yes')
```

В результате обучения модели SVM были получены следующие метрики, которые представлены в таблице 2.

**Как подбирал параметр C...**

**Ответ на вопрос, может ли SVM ...**

Таблица 2: Метрики модели SVM для C = 1.0.

| **Метрика** | **Значение** |
|---|---|
| Accuracy | 0.91866 |
| Precision | 0.91866 |
| Recall | 1.0 |
| F1-score | 0.95761 |

## 2.2 Метрики многоклассовой классификации

### 2.2.1 AUC-ROC

#### 2.2.1.1 Определения

**AUROC** (Area Under the Receiver Operating Characteristic Curve), также известная как **AUC-ROC**, — это метрика качества бинарной классификации, которая оценивает способность модели классифицировать объекты независимо от выбранного порога отсечения.

По сути, AUC показывает вероятность того, что случайно выбранный положительный объект получит от модели более высокую оценку вероятности, чем случайно выбранный отрицательный:[5]

$$\mathrm{AUC}(Y, P) = \frac{\sum\limits_{i=1}^{N}\sum\limits_{j=1}^{N} \mathbb{I}[Y_i < Y_j]\mathbb{I}[P_i < P_j]}{\sum\limits_{i=1}^{N}\sum\limits_{j=1}^{N} \mathbb{I}[Y_i < Y_j]}, \tag{1}$$

где:

- $Y$ — массив истинных меток классов (0 для отрицательного, 1 для положительного);
- $P$ — массив предсказанных вероятностей принадлежности к положительному классу;
- $N$ — общее количество объектов в выборке;
- $\mathbb{I}$ — индикаторная функция. В данной формуле она используется в двух вариантах:
  - $\mathbb{I}[Y_i < Y_j]$ — стандартный строгий индикатор (равен 1, если $Y_i < Y_j$, и 0 в противном случае).
  - $\mathbb{I}[P_i < P_j]$ — индикатор сравнения вероятностей, который учитывает их возможное совпадение:

$$\mathbb{I}[P_i < P_j] = \begin{cases} 1, & P_i < P_j, \\ 1/2, & P_i = P_j, \\ 0, & P_i > P_j. \end{cases}$$

Такое дробное значение вводится, чтобы при равных выходных данных модель не получала ни преимущества, ни штрафа, что соответствует площади трапеции на ROC-кривой.

Обозначим $N_+$ — количество положительных объектов, а $N_-$ — количество отрицательных объектов в выборке.

**ROC-кривая** представляет собой график зависимости True Positive Rate от False Positive Rate при всех возможных порогах отсечения $t$: начинаем с $t = +\infty$, когда все предсказания равны 0, и двигаемся к $t = -\infty$. Поэтому мы сортируем все объекты по убыванию вероятности и последовательно двигаем порог:

- если встречаем положительный объект, то True Positive Rate увеличивается на $1/N_+$;
- если встречаем отрицательный объект, то False Positive Rate увеличивается на $1/N_-$.

AUC — это площадь под этой кривой, которая вычисляется как сумма площадей прямоугольников (или трапеций, при совпадении вероятностей), образованных шагами вправо.

Каждый раз, когда мы встречаем отрицательный объект $i$, мы делаем шаг вправо шириной $1/N_-$. Высота текущей ступеньки (TPR) в этот момент равна доле положительных объектов $j$, которые мы уже прошли (то есть тех, чей скор выше: $P_j > P_i$). Таким образом, вклад одного отрицательного объекта $i$ в общую площадь равен:

$$\Delta\text{AUC}_i = \frac{1}{N_-} \cdot \frac{\sum\limits_{j:Y_j=1} \mathbb{I}[P_j > P_i]}{N_+}.$$

Если просуммировать это по всем отрицательным объектам $i$ (у которых $Y_i = 0$), мы получим:

$$\text{AUC} = \sum_{i:Y_i=0} \Delta\text{AUC}_i = \frac{1}{N_+ \cdot N_-} \sum_{i:Y_i=0} \sum_{j:Y_j=1} \mathbb{I}[P_j > P_i].$$

Поскольку индикатор $\mathbb{I}[Y_i < Y_j]$ равен 1 только в случае, когда $Y_i = 0$ и $Y_j = 1$, мы можем переписать это в виде двойной суммы по всем объектам:

$$\text{AUC} = \frac{\sum\limits_{i=1}^{N} \sum\limits_{j=1}^{N} \mathbb{I}[Y_i < Y_j]\mathbb{I}[P_j > P_i]}{N_+ \cdot N_-}.$$

Заметим, что знаменатель $N_+ \cdot N_-$ в точности равен количеству всех возможных пар «отрицательный-положительный»:

$$N_+ \cdot N_- = \sum_{i=1}^{N} \sum_{j=1}^{N} \mathbb{I}[Y_i < Y_j]$$

Подставляя это в выражение, мы получаем в точности формулу (1).

Изначально метрика разработана для двух классов, поэтому её нельзя применять в многоклассовой классификации напрямую. Пусть $C$ — общее количество классов, $n_i$ — количество истинных экземпляров в классе $i$, а $N = \sum_{i=1}^{C} n_i$ — общее количество экземпляров в выборке. Тогда:

- **Macro-average** — простое среднее арифметическое AUC, посчитанных для каждого класса. Подходит для оценки того, насколько хорошо модель справляется с редкими классами, потому что каждый класс вносит одинаковый вклад в итоговую метрику:

$$\mathrm{AUC}_{\mathrm{macro}} = \frac{1}{C} \sum_{i=1}^{C} \mathrm{AUC}_i \tag{2}$$

- **Weighted-average** — среднее значение AUC по всем классам, взвешенное по количеству истинных экземпляров в каждом классе. Этот подход учитывает дисбаланс классов: классы с бóльшим количеством примеров вносят больший вклад в итоговую метрику:

$$\mathrm{AUC}_{\mathrm{weighted}} = \sum_{i=1}^{C} \frac{n_i}{N} \mathrm{AUC}_i \tag{3}$$

- **Micro-average** — агрегирует вклад каждого отдельного объекта данных, а не класса. Математически для AUC в многоклассовой постановке это сводится к вычислению единой бинарной ROC-кривой на основе преобразования матриц предсказаний всех классов в вектор.

  Если обозначить через $Y_{\mathrm{flat}}$ и $P_{\mathrm{flat}}$ развёрнутые массивы истинных меток и предсказанных вероятностей размерности $(N \times C)$, то:

$$\mathrm{AUC}_{\mathrm{micro}} = \mathrm{AUC}(Y_{\mathrm{flat}}, P_{\mathrm{flat}}) \tag{4}$$

- **Samples-average** — вычисляет метрику AUC отдельно для каждого объекта и затем усредняет полученные значения.

  Если транспонировать матрицы $Y$ и $P$, то их столбцы станут строками. Тогда метрика вычисляется как среднее бинарных AUC для каждого объекта:

$$\mathrm{AUC}_{\mathrm{samples}} = \frac{1}{N} \sum_{r=1}^{N} \mathrm{AUC}(Y_r^T, P_r^T), \tag{5}$$

  где $Y_r^T$ и $P_r^T$ — $r$-е столбцы транспонированных матриц (или $r$-е строки исходных матриц). Это полезно, если важно оценить, насколько хорошо модель ранжирует классы именно для каждого отдельного объекта *(например, в рекомендательных системах)*.

#### 2.2.1.2 Ручной расчёт характеристик

На слайде 43 приведены следующие матрицы для трёх классов:

$$Y = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad P = \begin{bmatrix} 0.7 & 0.6 & 0.5 \\ 0.3 & 0.4 & 0.6 \\ 0.5 & 0.9 & 0.2 \\ 0.4 & 0.5 & 0.1 \end{bmatrix}$$

Таблица 3: Результаты вычисления $\mathrm{AUC}_{\mathrm{macro}}$ для каждого класса.

| **Class** $i$ | $\mathbf{AUC}_i(Y_i, P_i)$ | $\mathbf{AUC}_{\mathbf{macro}}$ |
|---|---|---|
| 0 | 0.75 | 0.80556 |
| 1 | 0.66667 | |
| 2 | 1.0 | |

Таблица 4: Результаты вычисления $\mathrm{AUC}_{\mathrm{weighted}}$ для каждого класса.

| **Class** $i$ | $\mathbf{AUC}_i(Y_i, P_i)$ | $n_i$ | $\mathbf{AUC}_{\mathbf{weighted}}$ |
|---|---|---|---|
| 0 | 0.75 | 2 | 0.75 |
| 1 | 0.66667 | 3 | |
| 2 | 1.0 | 1 | |

*Macro-average AUC* вычисляется по формуле (2). Для этого сначала нужно рассчитать AUC для каждого класса по формуле (1). Результаты вычислений представлены в таблице 3.

*Weighted-average AUC* вычисляется по формуле (3). Для этого воспользуемся рассчитанными на предыдущем шаге AUC для каждого класса. Результаты вычислений представлены в таблице 4.

*Micro-average AUC* вычисляется по формуле (4):

$$\mathrm{AUC}_{\mathrm{micro}} = 0.83333$$

*Samples-average AUC* вычисляется по формуле (5). Для этого сначала нужно рассчитать AUC для каждого объекта по формуле (1). Результаты вычислений представлены в таблице 5.

#### 2.2.1.3 Визуализация ROC

В рамках детализации также рекомендуется визуализировать кривую ROC. Способ её построения подробно описан в разделе 2.2.1.1. Это выполнено при помощи пакета `matplotlib` на языке Python, полученный артефакт можно наблюдать на рисунке 1.

Таблица 5: Результаты вычисления $\mathrm{AUC}_{\mathrm{samples}}$ для каждого объекта.

| **Object** $r$ | $\mathbf{AUC}_r(Y_r^T, P_r^T)$ | $\mathbf{AUC}_{\mathbf{samples}}$ |
|---|---|---|
| 0 | 1.0 | 0.875 |
| 1 | 1.0 | |
| 2 | 1.0 | |
| 3 | 0.5 | |

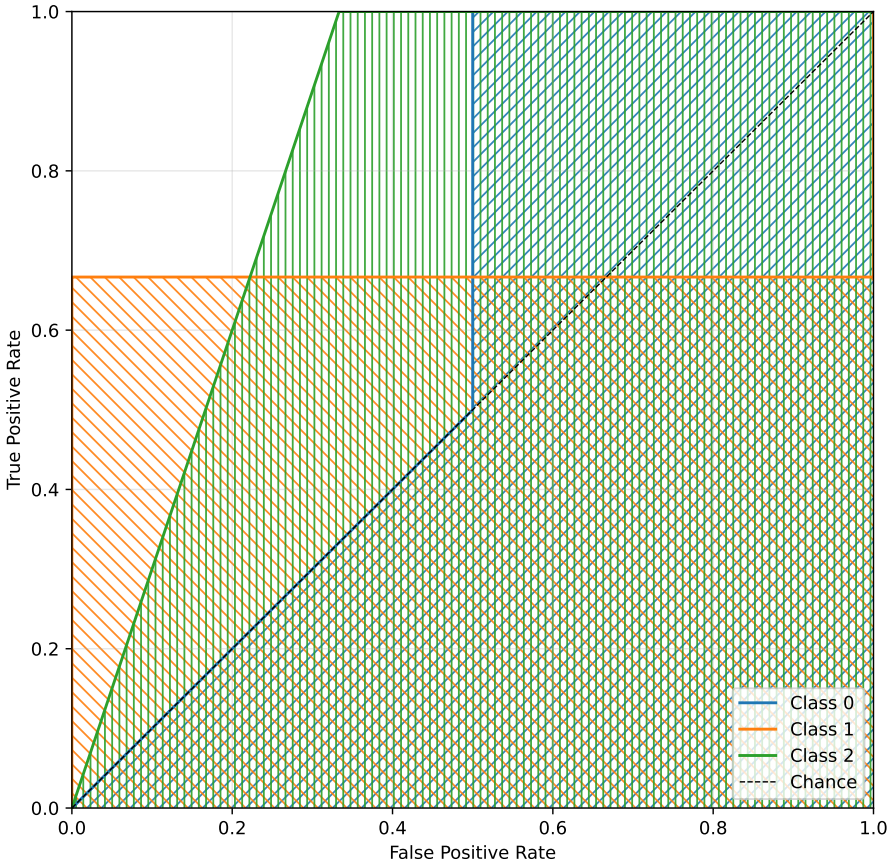



Рис. 1: Визуализация кривых ROC для трёх классов.

### 2.2.2 PR-кривая

#### 2.2.2.1 Определения

**Accuracy** — метрика, которая показывает долю верно классифицированных объектов относительно всех объектов:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}}, \tag{6}$$

где:

- **TP** (True Positive) — количество верно классифицированных объектов положительного класса;
- **TN** (True Negative) — количество верно классифицированных объектов отрицательного класса;
- **FP** (False Positive) — количество неверно классифицированных объектов отрицательного класса как положительные;
- **FN** (False Negative) — количество неверно классифицированных объектов положительного класса как отрицательные.

**Recall** *(True Positive Rate)* — метрика, которая показывает долю верно классифицированных объектов положительного класса относительно всех объектов положительного класса:

$$\text{TPR} = \text{R} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{7}$$

**Precision** *(Positive Predictive Value)* — метрика, которая показывает долю верно классифицированных объектов положительного класса относительно всех модельных объектов положительного класса:

$$\text{PPV} = \text{P} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{8}$$

#### 2.2.2.2 Ручной расчёт характеристик

Для проведения расчётов сфокусируемся на первом классе. Введём следующие пороги бинаризации (binary thresholds):

$$\text{bt} = \{0.7,\ 0.45,\ 0.1\}$$

В результате ручных вычислений по формулам (6), (7) и (8) были получены значения метрик, которые представлены в таблице 6. Их можно визуализировать на эскизе PR-кривой, который представлен на рисунке 2.

### 2.2.3 Выводы

...

## 2.3 Применение OneClassSVM

...

Таблица 6: Результаты вычисления метрик PR-кривой для первого класса.

| **Binary Threshold** | **Accuracy** | **Recall** | **Precision** |
|---|---|---|---|
| 0.7 | 0.5 | 0.66667 | 1.0 |
| 0.45 | 0.5 | 0.33333 | 0.66667 |
| 0.1 | 0.75 | 0.0 | 0.75 |

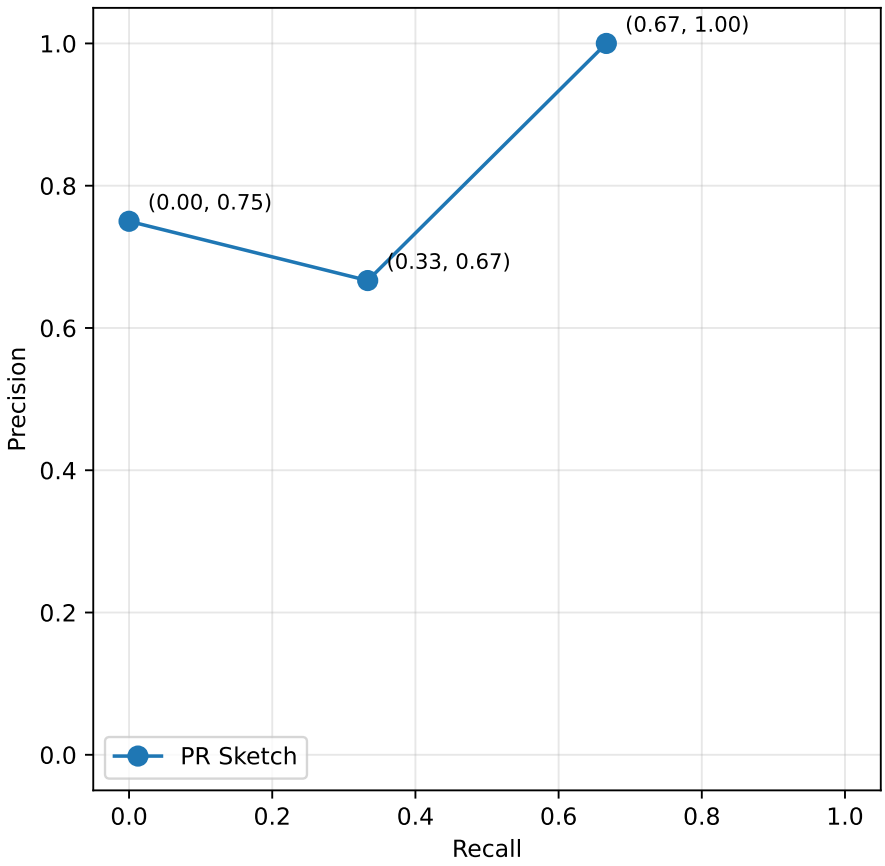



Рис. 2: Визуализация эскиза PR-кривой для первого класса.

# 3 Источники литературы


1. *scikit-learn developers.* Outlier detection / scikit-learn. — URL: https://scikit-learn.ru/stable/modules/outlier_detection.html (дата обр. 10.10.2026) ; Неофициальный перевод сайта scikit-learn.org.
2. *Дьяконов А.* Поиск аномалий / WordPress. — URL: https://alexanderdyakonov.wordpress.com/ (дата обр. 10.10.2026) ; Блог.
3. *Cortez P.* Student Performance Dataset / UCI Machine Learning Repository. — 2014. — URL: https://archive.ics.uci.edu/dataset/320/student+performance (дата обр. 24.09.2026).
4. *Лабин М. А.* Лабораторная работа №1: Pandas, EDA, линейная регрессия. — Санкт-Петербург, 2026. — Отчет по дисциплине «Системы искусственного интеллекта».
5. *Pandey R.* Lecture on Parametric and Non-Parametric Tests: Chi-Square test, ANOVA, Mann-Whitney, Kruskal Wallis and Kolmogrov-Smirnov / Department of Statistics, University of Lucknow. — 2020. — URL: https://www.lkouniv.ac.in/site/writereaddata/siteContent/202003241550010409rajeev_pandey_Non-Parametric.pdf (дата обр. 02.11.2026).